

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Edytor schematów szydełkowych z automatycznym rozmieszczaniem symboli

Etap 1: Specyfikacja wymagań i określenie zakresu projektu

Autorzy:

Anna Babicka, Wiktor Murawski

Promotor:

dr inż. Paweł Kotowski

20 października 2026

Spis treści

1 Historia zmian dokumentu **3**

2 Opis rozwiązania i motywacja **4**
2.1 Cel projektu 4
2.2 Motywacja 4
2.3 Użytkownicy docelowi 4
2.4 Proponowane podejście 4

3 Słownik **5**
3.1 Szydełkowanie 5
3.2 Model i algorytmy 6
3.3 Technologie i jakość 6
3.4 Dokumentacja i zarządzanie projektem 7

4 Zakres projektu: MVP i opcjonalne funkcjonalności **8**
4.1 Minimum Viable Product (MVP – Passing Scope) 8
4.2 Opcjonalne funkcjonalności 8

5 Wymagania funkcjonalne **9**

6 Mierzalne wymagania niefunkcjonalne (NFRs – Quantifiable Non-Functional Requirements) **14**

7 Zarządzanie ryzykiem – analiza SWOT **16**

8 Harmonogram i macierz RACI **17**
8.1 Macierz odpowiedzialności (RACI) 17

1 Historia zmian dokumentu

Wersja	Data	Autor	Opis zmian
1.0	18.10.2026	Anna Babicka	*Pierwsza wersja dokumentu*
1.1	19.10.2026	Anna Babicka	*Wprowadzenie poprawek po konsultacji z promotorem*
1.2	20.10.2026	Wiktor Murawski	*Rozwinięcie opisu rozwiązania i motywacji, rozwinięcie słownika, dodanie diagramu przypadków użycia, poprawa literówek*

2 Opis rozwiązania i motywacja

2.1 Cel projektu

Celem projektu jest zaprojektowanie, implementacja i ocena intuicyjnej aplikacji webowej do tworzenia schematów szydełkowych 2D. Aplikacja odciąża użytkownika od ręcznego rozmieszczania symboli: wzór jest reprezentowany jako graf oczek, a układ schematu wyznaczany jest automatycznie z uwzględnieniem geometrii oczek, takiej jak proporcje symboli, ich orientacja względem oczka, w które są wbite, oraz odległości między sąsiednimi oczkami. Dzięki temu czytelne i spójne ze strukturą wzoru schematy mają powstawać szybko i bez konieczności ręcznych poprawek.

2.2 Motywacja

Schemat jest wygodną, niezależną od języka formą zapisu wzoru, ponieważ bezpośrednio pokazuje relacje między oczkami. Jego tworzenie w dostępnych narzędziach pozostaje jednak w dużej mierze pracą ręczną. Podstawowe rozwiązania opierają się na regularnej siatce, a bardziej zaawansowane na rozmieszczaniu symboli na okręgach, co nie wystarcza dla wzorów o nieregularnym kształcie. Użytkownik musi wówczas samodzielnie przesuwać i obracać symbole, a każda zmiana struktury wzoru może wymagać ponownego dopasowania wielu wcześniej ustawionych elementów.

Ten sam problem pojawia się przy projektowaniu i modyfikowaniu wzorów. Trudno przewidzieć, jak zmiana wpłynie na kształt całej robótki, a sprawdzenie każdego wariantu na szydełku wymaga wielokrotnego prucia i szydełkowania. Narzędzie, które po każdej zmianie struktury natychmiast przelicza i pokazuje schemat, pozwala szybko porównywać warianty bez wykonywania robótki.

2.3 Użytkownicy docelowi

Aplikacja jest przeznaczona dla osób projektujących wzory szydełkowe: hobbystów tworzących własne wzory lub modyfikujących istniejące oraz autorów, którzy chcą opublikować lub wydrukować czytelny schemat swojego wzoru. Zakładamy, że użytkownik zna podstawowe ściegi i terminologię szydełkowania, natomiast nie jest wymagana wiedza o grafach ani algorytmach rozmieszczania. Aplikacja działa w przeglądarce, więc nie wymaga instalacji.

2.4 Proponowane podejście

Rozwiązanie opiera się na rozdzieleniu logicznej struktury wzoru od jego wizualnej reprezentacji. Użytkownik buduje wzór, dodając oczka i określając, w które elementy są wbite, a aplikacja na tej podstawie automatycznie wyznacza położenie i orientację symboli. Nie zakładamy opracowania nowego algorytmu. Wartość inżynierską projektu stanowi dobór, integracja i dostosowanie istniejących metod (w szczególności algorytmów typu force-directed) do potrzeb interaktywnego edytora. Kluczowym kryterium oceny będzie praktyczna użyteczność: czy aplikacja pozwala uzyskać poprawny schemat szybciej niż ręczne rozmieszczanie symboli w ogólnym narzędziu graficznym.

3 Słownik

3.1 Szydełkowanie

Termin	Definicja
Oczko (ścieg, stitch)	Podstawowa jednostka robótki, powstająca przez przeciągnięcie nici przez pętle na szydełku.
Łańcuszek (ch)	Oczko niewbijane w robótkę. Podnosi poziom pracy i tworzy łuki.
Oczko ścisłe (sl st)	Oczko łączące dwa punkty bez zwiększania wysokości. Służy m.in. do zamykania okrążeń.
Półsłupek (sc), **słupek półpodwójny (hdc),** **słupek (dc),** **słupek podwójny (tr)**	Podstawowe oczka różniące się wysokością.
Wbicie (punkt wbicia, insertion)	Miejsce, w które wkłada się szydełko przy tworzeniu oczka, np. pod obie pętle, przednią lub tylną pętlę albo w łuk.
Magiczne kółko (mr)	Technika rozpoczęcia robótki okrężnej z zaciąganą pętlą.
Okrążenie / rząd	Grupa oczek tworzonych sukcesywnie, wykonywana po okręgu lub w linii.
Warstwa	Reprezentacja okrążenia lub rzędu w modelu aplikacji. Model nie rozróżnia tych dwóch technik; różnią się one jedynie sposobem przejścia do kolejnej warstwy.
Dodawanie / łączenie (increase / decrease)	Zwiększanie lub zmniejszanie liczby oczek w warstwie.
Schemat (chart)	Graficzny zapis wzoru za pomocą standardowych symboli.
Próbka (swatch)	Fragment robótki o znanej liczbie oczek, używany do kalibracji wymiarów.

3.2 Model i algorytmy

Termin	Definicja
Graf wzoru	Reprezentacja wzoru, w której wierzchołki to oczka, a krawędzie to relacje między nimi.
Układ (layout)	Przypisanie symbolom pozycji, obrotów i skal na płaszczyźnie.
Algorytm force-directed	Metoda wyznaczania układu przez symulację sił przyciągania i odpychania między wierzchołkami.
Stress / stress majorization	Miara i metoda minimalizacji rozbieżności między odległościami na rysunku a odległościami docelowymi.
Proporcja H/W (aspect ratio)	Stosunek wysokości do szerokości oczka, z wartością preferowaną i granicami.
Kalibracja	Dopasowanie parametrów geometrii do włóczki i napięcia użytkownika na podstawie próbki.
Seed (ziarno)	Wartość początkowa generatora liczb pseudolosowych. Ten sam seed i dane wejściowe dają ten sam wynik algorytmu.

3.3 Technologie i jakość

Termin	Definicja
SVG	Wektorowy format grafiki, używany do renderowania i eksportu schematów.
Undo/redo	Cofanie i ponawianie operacji.
Web Worker	Mechanizm przeglądarki do wykonywania obliczeń w osobnym wątku.
MVP	Minimalna wersja produktu realizująca podstawowe funkcje.
NFR	Wymaganie niefunkcjonalne, dotyczące jakości, a nie funkcji systemu.
Percentyl (p95)	Wartość, poniżej której mieści się 95% pomiarów.
JSON	Tekstowy format wymiany danych, używany do zapisu i wczytywania projektu.
Paleta symboli	Panel interfejsu, z którego użytkownik wybiera typ oczka do dodania.
FPS (frames per second)	Liczba klatek obrazu wyświetlanych na sekundę. Miara płynności interfejsu.

3.4 Dokumentacja i zarządzanie projektem

Termin	Definicja
Historyjka użytkownika (US, User Story)	Krótki opis funkcji z perspektywy użytkownika w formie: jako ..., chcę ..., aby ....
Kryteria akceptacji	Warunki, które musi spełniać funkcja, aby uznać historyjkę za zrealizowaną.
Gherkin	Format zapisu kryteriów akceptacji w postaci scenariuszy Given (stan początkowy), When (akcja), Then (oczekiwany wynik).
SWOT	Analiza mocnych i słabych stron projektu oraz szans i zagrożeń z jego otoczenia.
RACI	Macierz przypisująca zadaniom role: wykonawca (R), odpowiedzialny (A), konsultowany (C), informowany (I).

4 Zakres projektu: MVP i opcjonalne funkcjonalności

Zakres projektu został podzielony na MVP (Minimum Viable Product) - kluczowe funkcjonalności, które muszą działać, żeby móc uznać aplikację za funkcjonalną, oraz opcjonalne funkcjonalności, które poszerzają możliwości aplikacji, ale nie są niezbędne i mogą zostać zaimplementowane, jeśli starczy na to czasu.

4.1 Minimum Viable Product (MVP – Passing Scope)

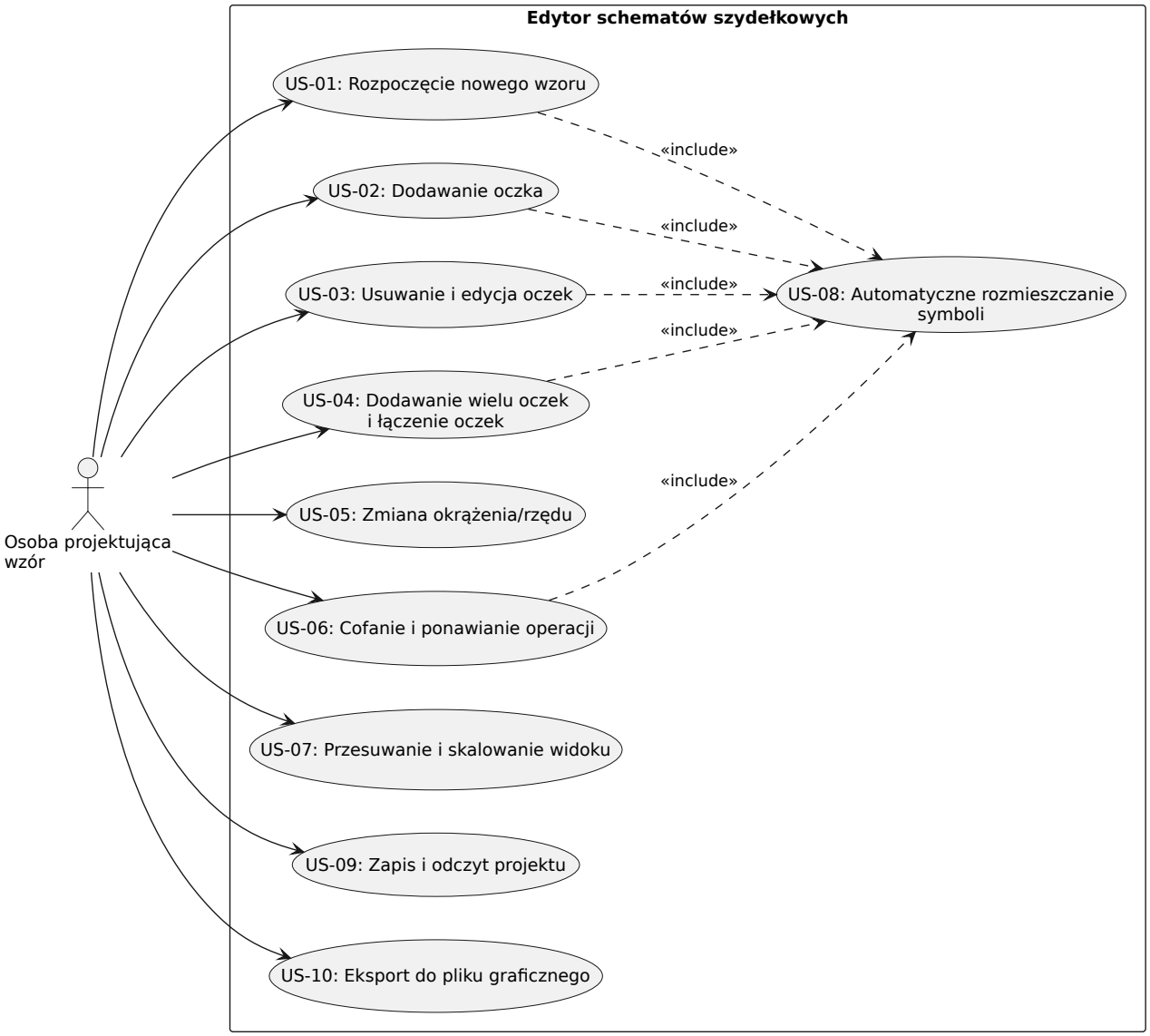
1. **Graficzny interfejs edytora:** Pozwalający na łatwe i intuicyjne dodawanie, usuwanie i edycję elementów oraz wyświetlanie obecnego stanu wzoru.
2. **Automatyczne rozmieszczanie symboli:** Wyznaczenie wzajemnego położenia symboli odwzorowującego rzeczywiste ich rozmieszczenie po wyszydełkowaniu wzoru za pomocą algorytmu force-directed.
3. **Zapisanie i wczytanie postępów pracy:** Eksport i import danych schematu do pliku np. w formacie JSON, zachowującego informacje strukturalne.
4. **Eksport schematu do pliku w formacie graficznym:** Zapisanie gotowego schematu do pliku SVG, konwersja do PDF, PNG, JPEG.

4.2 Opcjonalne funkcjonalności

1. Możliwość ręcznych poprawek ustawienia poszczególnych symboli / grup symboli, w tym zablokowanie pozycji wybranych elementów.
2. Modyfikacja parametrów algorytmu automatycznego rozmieszczania.
3. Identyfikacja i wizualne zaznaczenie potencjalnie problematycznych fragmentów, mogących świadczyć o błędach we wzorze.
4. Konwersja schematu do instrukcji tekstowych i na odwrót.
5. Grupowanie elementów i możliwość dodawania/usuwania całych grup symboli.
6. Wizualizacja kolejności wykonywania elementów przy odtwarzaniu wzoru.

5 Wymagania funkcjonalne

Wymagania funkcjonalne zostały zebrane w formie historyjek użytkownika (User Stories) i przedstawione na diagramie przypadków użycia. Wymagania dotyczą tylko zakresu MVP, opcjonalne funkcjonalności nie zostały tutaj uwzględnione. W tym przypadku mamy tylko jednego użytkownika - osobę projektującą wzór.



Rysunek 1: Diagram przypadków użycia

US-01: Rozpoczęcie nowego wzoru

Jako: Osoba projektująca wzór,
Chcę: utworzyć nowy wzór, wybierając sposób jego rozpoczęcia,
Aby: mieć od czego zacząć dodawanie kolejnych oczek.

Kryteria akceptacji (Gherkin):

Scenariusz: rozpoczęcie wzoru
Given: w edytorze otwarty jest pusty wzór
When: użytkownik wybiera typ elementu początkowego i podaje jego parametry (zgodnie z tabelą elementów początkowych)
Then: do wzoru zostają dodane elementy początkowe, pozycja nici wskazuje ostatni z nich, a bieżąca warstwa ma numer 1

Scenariusz: nowy wzór przy niezapisanych zmianach
Given: bieżący wzór ma niezapisane zmiany
When: użytkownik wybiera opcję „Nowy wzór"
Then: wyświetla się potwierdzenie utraty zmian, a po jego odrzuceniu bieżący wzór pozostaje niezmieniony

US-02: Dodawanie oczka przez kliknięcie

Jako: Osoba projektująca wzór,
Chcę: wybrać typ elementu z palety symboli i dodać go, klikając na element, w który chcę go wbić,
Aby: tworzyć wzór zgodnie z tym, jak powinien być on wyszydełkowany.

Kryteria akceptacji (Gherkin):

Scenariusz: dodanie oczka
Given: wzór zawiera co najmniej jeden element, a w palecie wybrany jest słupek
When: użytkownik klika symbol na schemacie
Then: do wzoru zostaje dodany słupek wbity w kliknięty element, a na schemacie pojawia się nowy symbol.

Scenariusz: niedozwolone wbicie
Given: wzór zawiera co najmniej jeden element, a w palecie wybrany jest typ oczka
When: użytkownik klika element, w który oczka tego typu nie można wbić (zgodnie z regułami modelu)
Then: wzór pozostaje bez zmian i wyświetla się komunikat z przyczyną

US-03: Usuwanie i edycja oczek

Jako: Osoba projektująca wzór,
Chcę: usuwać oczka i zmieniać ich właściwości,
Aby: poprawiać błędy bez budowania wzoru od nowa.

Kryteria akceptacji (Gherkin):

Scenariusz: usunięcie ostatniego oczka
Given: wzór zawiera co najmniej jedno oczko
When: użytkownik wybiera "Usuń ostatnie oczko"
Then: ostatnio dodane oczko zostaje usunięte razem z jego połączeniami i pozycja nici wraca do poprzedniego oczka

Scenariusz: zmiana typu oczka
Given: użytkownik zaznaczył oczko

When: wybiera inny typ oczka w panelu właściwości, np. słupek zamiast półsłupka
Then: oczko zmienia typ, a jego połączenia pozostają bez zmian i układ symboli zostaje przeliczony

US-04: Dodawanie i łączenie oczek

Jako: Osoba projektująca wzór,
Chcę: dodawać kilka oczek w jedno miejsce i łączyć oczka,
Aby: zmieniać kształt robótki.

Kryteria akceptacji (Gherkin):

Scenariusz: dodawanie oczek w jedno miejsce
Given: w palecie wybrany jest typ oczka pozwalający na wbicie
When: użytkownik klika to samo oczko kilka razy z rzędu
Then: każde kliknięcie dodaje nowe oczko wybranego typu wbity w to samo oczko i symbole rozkładają się wachlarzowo nad oczkiem bazowym

Scenariusz: łączenie kilku oczek
Given: w palecie wybrany jest typ oczka pozwalający na wbicie, a tryb łączenia jest włączony
When: użytkownik zaznacza kolejno dwa lub więcej sąsiednich oczek i zatwierdza wybór
Then: zostaje dodany jedno oczko wybranego typu wbite we wszystkie zaznaczone oczka, a jego symbol odzwierciedla łączenie

US-05: Zmiana okrążenia/rzędu

Jako: Osoba projektująca wzór,
Chcę: zakończyć okrążenie/rząd i rozpocząć następne,
Aby: budować wzór warstwami i zwiększyć jego czytelność.

Kryteria akceptacji (Gherkin):

Scenariusz: rozpoczęcie nowej warstwy
Given: bieżąca warstwa ma numer n i zawiera co najmniej jeden element
When: użytkownik wybiera opcję "nowa warstwa"
Then: numer bieżącej warstwy wynosi *n+1* i dodawane elementy mają przypisany numer warstwy *n+1*.

US-06: Operacje undo/redo

Jako: Osoba projektująca wzór,
Chcę: cofać i ponawiać operacje,
Aby: swobodnie eksperymentować.

Kryteria akceptacji (Gherkin):

Scenariusz: cofnięcie operacji
Given: użytkownik wykonał operację modyfikującą wzór

When: naciska Ctrl+Z lub przycisk "Cofnij"
Then: wzór, pozycja nici i numer okrążenia wracają do stanu sprzed operacji

Scenariusz: ponowienie operacji
Given: użytkownik cofnął operację
When: naciska Ctrl+Y lub przycisk "Ponów"
Then: wzór wraca do stanu po tej operacji

US-07: Przesuwanie i skalowanie wzoru

Jako: Osoba projektująca wzór,
Chcę: przesuwać i przybliżać schemat,
Aby: wygodnie pracować zarówno z detalami, jak i z całym wzorem.

Kryteria akceptacji (Gherkin):

Scenariusz: przybliżanie i oddalanie
Given: schemat jest wyświetlony
When: użytkownik obraca kółkiem myszy albo klika "+"lub "-"
Then: widok przybliża się lub oddala względem pozycji kursora albo środka widoku i aktualna skala jest widoczna w interfejsie

Scenariusz: przesuwanie widoku
Given: schemat jest wyświetlony
When: użytkownik przeciąga tło
Then: widok przesuwa się zgodnie z ruchem kursora

US-08: Automatyczne rozmieszczanie symboli

Jako: Osoba projektująca wzór,
Chcę: aby symbole układały się automatycznie po wprowadzeniu zmian,
Aby: uniknąć konieczności ręcznego ustawiania i obracania symboli.

Kryteria akceptacji (Gherkin):

Scenariusz: przeliczenie układu po zmianie
Given: wzór jest wyświetlony na schemacie
When: użytkownik modyfikuje wzór
Then: układ symboli zostaje przeliczony i wyświetlają się one w zaktualizowanych pozycjach

Scenariusz: orientacja i proporcje symboli
Given: układ został przeliczony
When: użytkownik ogląda schemat
Then: każdy symbol jest skierowany podstawą w stronę oczka, w które jest wbity, proporcje H/W każdego symbolu mieszczą się w granicach zdefiniowanych dla jego typu i symbole nie nakładają się na siebie (na tyle na ile to możliwe)

Scenariusz: duża liczba oczek
Given: wzór zawiera ok. 500 oczek

When: trwa przeliczanie układu
Then: interfejs reaguje na działania użytkownika (przewijanie, przybliżanie, wybór w palecie)

US-09: Zapis i odczyt

Jako: Osoba projektująca wzór,
Chcę: zapisać projekt do pliku i ponownie go otworzyć,
Aby: kontynuować pracę później.

Kryteria akceptacji (Gherkin):

Scenariusz: zapis i ponowne wczytanie
Given: użytkownik utworzył wzór
When: zapisuje projekt do pliku, a następnie go wczytuje
Then: wzór, pozycja nici i numer bieżącego okrążenia pozostają takie same jak przed zapisem

Scenariusz: niepoprawny plik
Given: w edytorze jest otwarty wzór
When: użytkownik wczytuje uszkodzony lub niepoprawny plik
Then: wyświetla się komunikat błędu, a bieżący wzór pozostaje niezmieniony

Scenariusz: niezapisane zmiany
Given: wzór ma niezapisane zmiany
When: użytkownik próbuje zamknąć lub odświeżyć stronę
Then: pojawia się ostrzeżenie o utracie zmian

US-10: Eksport do pliku graficznego

Jako: Osoba projektująca wzór,
Chcę: wyeksportować schemat do pliku SVG, PDF, PNG lub JPEG,
Aby: go wydrukować lub opublikować.

Kryteria akceptacji (Gherkin):

Scenariusz: eksport schematu Given: wzór zawiera co najmniej jedno oczko
When: użytkownik wybiera eksport do wybranego formatu
Then: w razie potrzeby wzór jest konwertowany do wybranego formatu, zostaje pobrany plik z całym schematem, niezależnie od bieżącego przybliżenia widoku.

6 Mierzalne wymagania niefunkcjonalne (NFRs – Quantifiable Non-Functional Requirements)

Poniższe wymagania niefunkcjonalne opisują oczekiwaną jakość działania systemu. Każde z nich ma mierzalną wartość docelową i określoną metodę weryfikacji. Podane progi są wartościami wstępnymi i mogą zostać skorygowane na podstawie pomiarów prototypu.

ID	Kategoria	Docelowa mierzalna metryka	Sposób weryfikacji
NFR-P1	**Wydajność**	Dodanie oczka jest widoczne na schemacie w $\leq$ 200 ms (p95) dla wzoru do 500 oczek.	Pomiar czasu od kliknięcia do renderu, np. `performance.now()` lub React Profiler
NFR-P2	**Wydajność**	Pełne przeliczenie układu dla wzoru 500 oczek trwa $\leq$ 2 s.	Benchmark na zestawie testowych wzorów
NFR-P3	**Wydajność**	Podczas przeliczania układu interfejs utrzymuje $\geq$ 30 FPS przy przewijaniu i przybliżaniu.	Chrome DevTools, zakładka Performance
NFR-P4	**Wydajność**	Pierwsze załadowanie aplikacji: Largest Contentful Paint $\leq$ 2,5 s.	Chrome DevTools Lighthouse
NFR-L1	**Jakość układu**	95% symboli ma proporcje H/W w zdefiniowanych granicach.	Automatyczny test na zestawie wzorów
NFR-L2	**Jakość układu**	Pole nakładania się symboli wynosi $\leq$ 1% ich łącznego pola.	Automatyczny test geometryczny
NFR-L3	**Jakość układu**	Ten sam wzór daje ten sam układ (deterministyczność przy stałym seedzie).	Test porównujący dwa uruchomienia
NFR-C1	**Kompatybilność**	Aplikacja działa poprawnie (wszystkie scenariusze MVP przechodzą) w przeglądarkach desktopowych w wersjach nie starszych niż: Chrome 155, Edge 155, Firefox 157, Safari 26 (macOS). Stan wersji na dzień 19.10.2026.	Testy manualne lub Playwright na wielu przeglądarkach
NFR-C2	**Kompatybilność**	Interfejs jest używalny od rozdzielczości 1280×720.	Testy wizualne

NFR-R1	**Niezawodność**	Zapis i wczytanie projektu zachowuje 100% danych wzoru.	Test automatyczny: zapis, odczyt, porównanie
NFR-R2	**Niezawodność**	Niepoprawny plik projektu nigdy nie powoduje awarii aplikacji.	Testy z uszkodzonymi plikami (fuzzing JSON)
NFR-R3	**Niezawodność**	Niezapisany stan jest odtwarzany po przypadkowym odświeżeniu strony (autozapis co ≤ 30 s).	Test manualny
NFR-M1	**Utrzymywalność**	Pokrycie testami modułu modelu i algorytmów $\geq 80\%$.	Raport pokrycia, np. Vitest z istanbul

7 Zarządzanie ryzykiem – analiza SWOT

Analiza SWOT przedstawia mocne i słabe strony projektu (czynniki wewnętrzne) oraz szanse i zagrożenia wynikające z jego otoczenia (czynniki zewnętrzne).

	Pozytywne	**Negatywne**
Wewnętrzne	**Mocne strony (S):** - Nowoczesny, dobrze udokumentowany stos (React, TypeScript, SVG) - Architektura z oddzielonym modelem, łatwa do testowania - Nacisk na UX, którego brakuje istniejącym narzędziom	**Słabe strony (W):** - Ograniczone doświadczenie z algorytmami layoutu grafów - Niepewne dane o geometrii oczek - Ograniczony czas i dwuosobowy zespół - Trudność obiektywnej oceny "ładności"układu - Zależność dwóch części pracy od wspólnego interfejsu
Zewnętrzne	**Szanse (O):** - Niszowy obszar z małą liczbą narzędzi łączących automatyczny układ z edycją - Duża społeczność szydełkująca i rynek wzorów - Gotowe biblioteki (d3-force, WebCola) skracające implementację - Możliwość dalszego rozwoju: instrukcje tekstowe, 3D, kalibracja - Rosnące zainteresowanie badawcze cyfryzacją rękodzieła (prace z 2022–2025)	**Zagrożenia (T):** - Istniejące narzędzia (CrochetPARADE, Stitch Fiddle) mogą rozwinąć podobne funkcje - Brak standardu notacji utrudnia uniwersalność - Ograniczony dostęp do użytkowników testowych - Ryzyko, że wynik nie spełni oczekiwań estetycznych osób szydełkujących - Zmiany wymagań ze strony promotora

8 Harmonogram i macierz RACI

[Tydzień 01-03] Etap 1: Rozpisanie wymagań, Podział na MVP i funkcje opcjonalne, Research dostępnych narzędzi i technologii, Analiza ryzyka.
[Tydzień 04] Etap 2: Zdefiniowanie modeli, architektury, komunikacji między modułami, Wybór algorytmu(-ów) do rozmieszczania symboli.
[Tydzień 05-06] Etap 3: Implementacja poszczególnych komponentów MVP, Testy jednostkowe.
[Tydzień 07-08] Etap 4: Integracja modułów, Testy integracyjne, Pierwsza funkcjonalna wersja aplikacji, Prezentacja aplikacji promotorowi i zebranie informacji o ewentualnych koniecznych poprawkach.
[Tydzień 09-11] Etap 5: Bufor zapasowy czasu w razie potrzeby, Inkrementalna implementacja, testowanie i integracja dodatkowych funkcjonalności.
[Tydzień 12] Zamrożenie kodu, Zakończenie prac nad aplikacją.
[Tydzień 13-14] Etap 6: Prezentacja finalnej wersji aplikacji promotorowi, Ewaluacja wydajności, Pisanie pracy dyplomowej
[Tydzień 15] Uzyskanie oceny od promotora, Przygotowanie wersji do wysłania do APD, Przygotowanie prezentacji na obronę

8.1 Macierz odpowiedzialności (RACI)

Definicje ról:

- **R (Responsible):** Osoba wykonująca zadanie (bezpośredni autor kodu / dokumentacji).
- **A (Accountable):** Osoba ponosząca ostateczną odpowiedzialność za jakość wyników i dotrzymanie terminów (**dokładnie 1 osoba na każde zadanie!**).
- **C (Consulted):** Ekspert merytoryczny, z którym konsultowano kwestie techniczne i który dokonał przeglądu.
- **I (Informed):** Osoba na bieżąco informowana o postępach i ostatecznych decyzjach.

Zadanie	**Anna**	**Wiktor**	**Promotor**
Cel, zakres, MVP, wymagania, harmonogram, zarządzanie ryzykiem (Etap 1)	**A**	R	C
Projekt architektury i interfejsów (Etap 2)	**A / R**	R	C
Przegląd literatury i dane o geometrii oczek (Etap 2)	**A / R**	C	I
Model danych i operacje na grafie (Etap 2)	**A / R**	C	C
Algorytm rozmieszczania i metryki estetyczne (Etap 2)	**A / R**	I	C
Projekt UX/UI (Etap 2)	C	**A / R**	I
Implementacja edytora (SVG, interakcje, undo/redo) (Etap 3)	C	**A / R**	-
Testy automatyczne i code review (Etap 3)	R	**A / R**	C
Integracja modułu algorytmicznego z edytorem (Etap 4)	**A / R**	R	-
Ewaluacja algorytmu (Etap 4)	**A / R**	C	C

Implementacja dodatkowych funkcjonalności (Etap 5)	R	**A / R**	C
Praca pisemna: wstęp, przegląd literatury, użyte algorytmy, model danych (Etap 6)	**A / R**	R	C
Praca pisemna: architektura rozwiązania, implementacja, wyniki (Etap 6)	R	**A / R**	C
Praca pisemna: Redakcja i zatwierdzenie całości (Etap 6)	R	R	**A**